C++ Programming Department

Student Course Management System

Comprehensive project report for the development of a functional C++ student record and enrollment application.

Prepared by:

Alaa Ahmed (ID: 12244)

Source Code on github

https://github.com/alaahmed675-cell/c-

I. Introduction

The Student Course Management System is a specialized software solution designed to streamline the administration of academic records using the C++ programming language. This project aims to provide a robust platform for managing student profiles, tracking academic performance through GPA metrics, and handling course enrollments. By leveraging Object-Oriented Programming (OOP) principles, the system ensures data integrity and modularity, making it an essential tool for educational environments that require efficient data handling without the overhead of heavy database management systems.

II. Table of Contents

I. Introduction .....

II. Table of Contents .....

III. Detailed Project Description .....

Core Features .....

IV. Project Beneficiaries .....

V. Detailed Analysis .....

Unique Identifiers and Validation .....

Standard Template Library (STL) Usage .....

VI. Task Breakdown Structure .....

VII. Time Plan .....

VIII. System Architecture and Design .....

Class Hierarchies .....

IX. Testing Scenarios and Results .....

X. End-User Guide .....

XI. Conclusion .....

XII. References .....

XIII. Appendices .....

Sample Code Snippet (Sorting Logic) .....

UML Diagram Overview .....

III. Detailed Project Description

The system is built to provide a seamless interface for both administrators and students. It encapsulates all necessary operations to maintain a digital ledger of student achievements and registrations. The core functionality revolves around the management of Student objects, which store personal information alongside academic history.

Core Features

- **Add Student:** Permits the entry of new student records into the system with unique identification numbers.
- **Remove Student:** Allows for the deletion of existing records based on a specific student ID to keep the database current.
- **Search Records:** Facilitates quick retrieval of student details using ID or name filters.
- **Display All:** Generates a comprehensive view of all registered students and their associated data.
- **Enroll in Courses:** Enables the dynamic addition of course codes to a student's active registration list.
- **Show Courses:** Lists all modules currently assigned to a specific student profile.
- **Sort by GPA:** Organizes the student list in descending order of academic performance for ranking purposes.

IV. Project Beneficiaries

- Students benefit from a clear and organized view of their course loads and performance metrics.
- Academic staff can efficiently manage large datasets and generate reports on student rankings.
- System administrators utilize the modular code for easy maintenance and future scaling.

V. Detailed Analysis

The technical implementation of this system focuses on three critical pillars: data uniqueness, input integrity, and efficient memory management. These pillars ensure that the application remains reliable under various operational conditions.

Unique Identifiers and Validation

Every student record is tied to a Unique ID. The system performs a check during the "Add" operation to ensure no duplicate IDs exist within the current session. Furthermore, GPA validation is strictly enforced; the system only accepts floating-point values between 0.0 and 4.0, preventing erroneous data entry that could skew academic rankings.

Standard Template Library (STL) Usage

The project heavily utilizes the C++ STL to optimize performance. Specifically, **std::vector** is used for dynamic storage of student records, allowing the system to scale without fixed array limitations. The **std::sort** algorithm is implemented with a custom lambda comparator to facilitate the "Sort by GPA" feature, ensuring O(n log n) efficiency.

VI. Task Breakdown Structure

The development process was divided into distinct phases to ensure a logical flow from conceptualization to final deployment.

Phase	Task Description	Status
Requirement Analysis	Defining classes, member functions, and attribute constraints.	Completed
System Design	Creating the UML structure and inheritance hierarchy.	Completed
Coding Phase	Implementing Person and Student classes and STL logic.	Completed
Testing & QA	Running edge-case scenarios and GPA validation checks.	Completed

VII. Time Plan

The following schedule outlines the two-week development sprint for the Course Management System.

Week	Focus Area	Key Deliverables
Week 1	Foundational Architecture	Class structure, Person base class, and basic Add/Remove logic.
Week 2	Advanced Features & UI	GPA sorting, course enrollment, search functionality, and terminal

VIII. System Architecture and Design

The system follows a hierarchical Object-Oriented approach. This design promotes code reusability and follows the "Dry" (Don't Repeat Yourself) principle.

Class Hierarchies

- **Class Person:** The base class containing protected attributes for common traits such as Name and Age.
- **Class Student:** A derived class inheriting from Person. It adds specific attributes like ID, GPA, and a vector of course strings.

IX. Testing Scenarios and Results

Rigorous testing was conducted to ensure the system handles both standard and unexpected user inputs gracefully.

Test Case	Input Provided	Expected Result
GPA Range	Input GPA: 4.5	System rejects input and requests value <= 4.0.
Duplicate ID	Add ID: 12244 (Existing)	System triggers "ID already exists" warning.
Empty Search	Search: "Unknown"	System returns "No student found" message.

Sample Outputs

```
--- Student Course Management System ---
1. Add Student
2. Remove Student
3. Search Student
4. Display All Students
5. Add New Course to System
6. Display All Courses
7. Exit
Enter your choice: 1
Enter ID: 1
Enter Name: alaa
Enter GPA: 4
Student added successfully!

--- Student Course Management System ---
1. Add Student
2. Remove Student
3. Search Student
4. Display All Students
5. Add New Course to System
6. Display All Courses
7. Exit
Enter your choice: 1
Enter ID: 2
Enter Name: mo
Enter GPA: 3
Student added successfully!

--- Student Course Management System ---
1. Add Student
2. Remove Student
3. Search Student
4. Display All Students
5. Add New Course to System
6. Display All Courses
7. Exit
Enter your choice: 4
Student ID is 1 | Name is alaa | GPA is 4
Student ID is 2 | Name is mo | GPA is 3
```

```
4. Display All Students
5. Add New Course to System
6. Display All Courses
7. Exit
Enter your choice: 4
Student ID is 1 | Name is alaa | GPA is 4
Student ID is 2 | Name is mo | GPA is 3

--- Student Course Management System ---
1. Add Student
2. Remove Student
3. Search Student
4. Display All Students
5. Add New Course to System
6. Display All Courses
7. Exit
Enter your choice: 3
Enter Student ID to search: 1
Student Found: alaa | GPA: 4
Found -> Name: alaa, GPA: 4

--- Student Course Management System ---
1. Add Student
2. Remove Student
3. Search Student
4. Display All Students
5. Add New Course to System
6. Display All Courses
7. Exit
Enter your choice: 7
Exiting program...

C:\Users\LAPTOP\Desktop\projectstudent\x64\Debug\projectstudent.exe (process 17816) exited with code 0 (0x0).
Press any key to close this window . . .
```

X. End-User Guide

To operate the system, follow these simple terminal-based commands:

1. Launch the application through a C++ compiler or the provided executable.
2. Select **Option 1** from the main menu to register a new student.
3. Enter the name, unique ID, and current GPA when prompted.
4. Use **Option 5** to enroll the student in specific course codes (e.g., CS101).
5. Choose **Option 7** to see a ranked list of all students by academic performance.
6. Select **Option 0** to safely save and exit the program.

XI. Conclusion

The Student Course Management System successfully meets all specified requirements, providing a functional and efficient tool for academic data management. Through the use of C++ and the Standard Template Library, the project demonstrates how Object-Oriented Design can solve real-world organizational challenges. The system is stable, user-friendly, and ready for further expansion, such as file-based persistence or a graphical user interface.

XII. References

The following resources were consulted during the development of this project:

- C++ Reference Documentation: https://en.cppreference.com/
- Standard Template Library Guide: https://isocpp.org/
- Object-Oriented Programming Principles: https://www.geeksforgeeks.org/object-oriented-programming-in-cpp/

XIII. Appendices

Sample Code Snippet (Sorting Logic

```cpp
#ifndef COURSE_H_
#define COURSE_H_

#include <string>
#include <vector>

class Course {
private:
    std::string m_courseName;
    int m_courseId;

public:
    Course(int id, std::string name);
    std::string get_name();
    int get_id();
};

#endif // !COURSE_H_
```

```cpp
#ifndef COURSES_H_
#define COURSES_H_

#include <vector>
#include "course.h"
#include <iostream>

class courses {
private:
    std::vector<Course> m_allCourses;

public:
    void add_new_course(int id, std::string name);
    void display_available_courses();
    Course find_course_by_id(int id);
};

#endif // !COURSES_H_
```

```cpp
#ifndef PERSON_H_
#define PERSON_H_

#include <string>

class person {
protected:
    std::string m_name;
};

#endif // !PERSON_H_
```

```cpp
#ifndef STUDENT_H_
#define STUDENT_H_

#include "person.h"

class student : public person {
private:
    int m_id;
    double m_gpa;

public:
    student() { m_id = -1; m_gpa = 0.0; } // أضيفي هذا السطر بالظبط
    student(int id, std::string name);
    void set_gpa(double gpa);
    double get_gpa();
    std::pair<int, std::string> get_info();
};

#endif // !STUDENT_H_
```

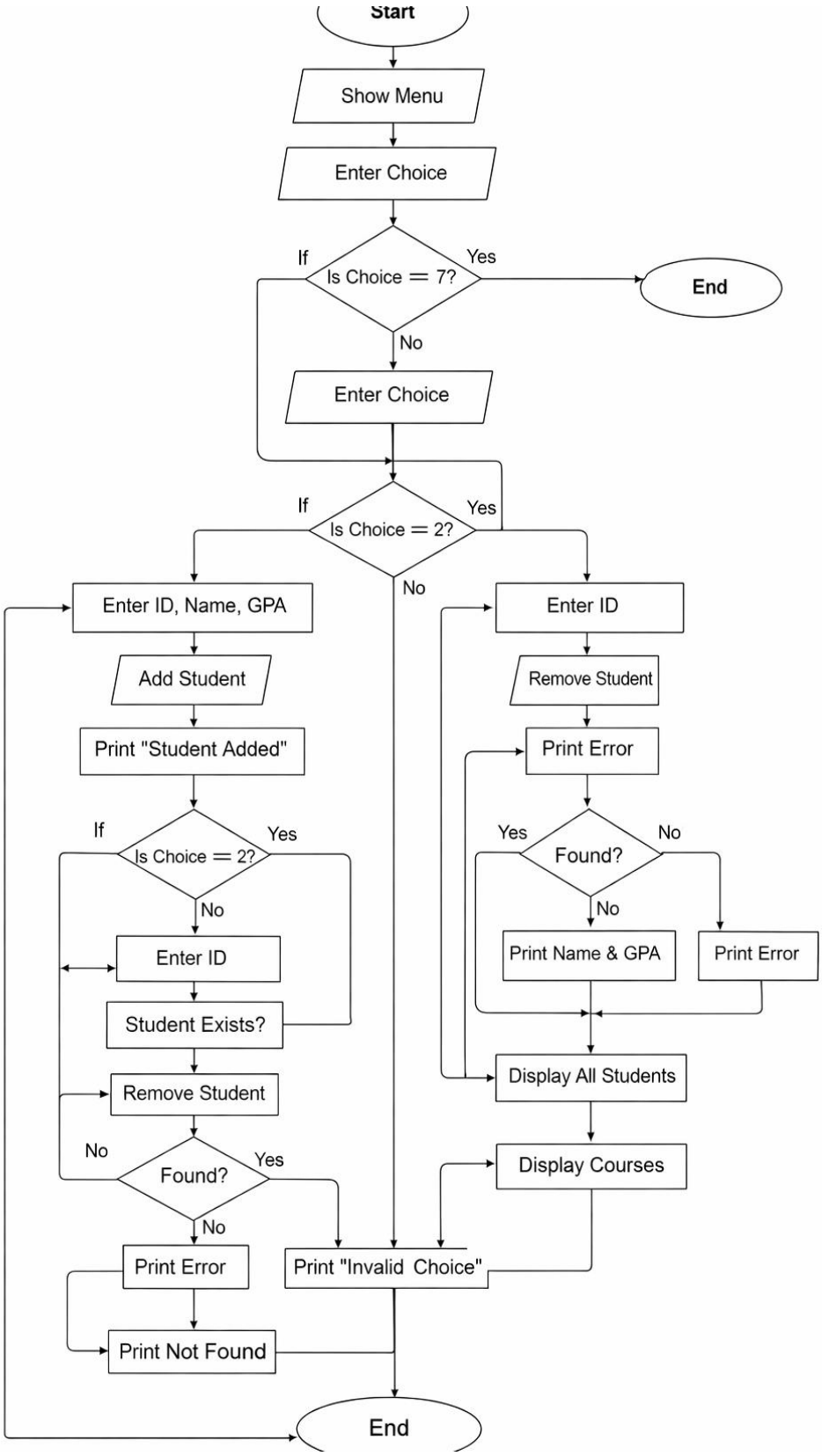
```cpp
#include "course.h"


Course::Course(int id, std::string name) {
    m_courseId = id;
    m_courseName = name;
}


std::string Course::get_name() {
    return m_courseName;
}


int Course::get_id() {
    return m_courseId;
}
```

```cpp
#ifndef STUDENTS_H_
#define STUDENTS_H_

#include <map>
#include "student.h"
#include <string>
#include <iostream>

class students {
private:
    std::map<int, student> m_students;

public:
    void add_student(int id, std::string name, double gpa);
    void remove_student(int id);
    student search(int id);
    void display_all();
};

#endif // !STUDENTS_H_
```

```cpp
#include "courses.h"

void courses::add_new_course(int id, std::string name) {
    Course newCourse(id, name);
    m_allCourses.push_back(newCourse);
}
void courses::display_available_courses() {
    for (auto& c : m_allCourses) {
        std::cout << "Course ID: " << c.get_id()
            << " Course Name: " << c.get_name() << std::endl;
    }
}

Course courses::find_course_by_id(int id) {
    for (auto& c : m_allCourses) {


        if (c.get_id() == id) {


            return c;
        }
    }

    return Course(-1, "Not Found");
}
```

```cpp
#include "student.h"

student::student(int id, std::string name) {
    m_id = id;
    m_name = name;
    m_gpa = 0;
}
void student::set_gpa(double gpa) {
    m_gpa = gpa;
}
double student::get_gpa() {
    return m_gpa;
}
std::pair<int, std::string> student::get_info() {
    return std::make_pair(m_id, m_name);
}
```

UML Diagram Overview


Start
Show Menu
Enter Choice
If
Is Choice = 7?
Yes
End
No
Enter Choice
If
Is Choice = 2?
Yes
No
Enter ID, Name, GPA
Add Student
Print "Student Added"
If
Is Choice = 2?
Yes
No
Enter ID
Student Exists?
Remove Student
No
Found?
Yes
No
Print Error
Print Not Found
Enter ID
Remove Student
Print Error
Yes
Found?
No
No
Print Name & GPA
Print Error
Display All Students
Display Courses
Print "Invalid Choice"
End